

Lesson 6

Reusable Components, Project Organization & Refactoring

Code the Dream — React

Today's Session

EXPLORE

- What makes a component reusable
- Variant props and the `children` prop
- Helper functions vs. custom hooks
- Project organization principles
- Safe refactoring workflow

APPLY

- Extract three components from a large component
- Refactor without breaking behavior
- Move filtering logic to a helper function
- Extract a custom hook and organize the project

By the end: identify when to extract a component, decide where logic belongs, and reorganize a project so it scales.

What Makes a Component Reusable?

A component is a good candidate for reuse when two conditions are met:

1. It accepts the **same props** in each use
2. It has the **same underlying structure** in each use

Common reusable patterns:

- List items — product cards, cart items, todo rows
- Layout wrappers — cards, modals, panels
- UI controls — buttons, form inputs, tooltips

If you find yourself copying and pasting JSX and only changing the data, that's a strong signal to extract a reusable component.

Variant Props — Configuring a Component's Appearance

A **variant prop** lets one component render in multiple visual styles based on a prop value.

Before — hardcoded

```
export function Dialog() {
  return (
    <div className="heading">
      <p>INFO dialog</p>
    </div>
  );
}
```

After — configurable

```
const colors = {
  error: '#f6bed7',
  info: '#bec7f6',
  success: '#bef6c5',
  warning: '#f6eebc',
};

export function Dialog({ kind = 'info' }) {
  return (
    <div style={{ backgroundColor: colors[kind] }}>
      <p>{kind.toUpperCase()} dialog</p>
    </div>
  );
}
```

The children Prop for Flexible Content

Use `children` when a component should wrap content it doesn't know about in advance.

```
// Component definition – {children} is an outlet
export function Dialog({ children, kind = 'info' }) {
  return (
    <div className="dialog">
      <div className="heading">...</div>
      <div className="content">{children}</div>
    </div>
  );
}
```

```
// Usage – anything between the tags becomes children
<Dialog kind="success">
  <h2>Order confirmed!</h2>
  <p>Your items will ship within 2 days.</p>
</Dialog>
```

`children` is better than a `message` prop when you want the caller to control the format — not just the text.

Check for Understanding

You're building a `Badge` component used in three places: a green "Active" badge, a red "Inactive" badge, and a yellow "Pending" badge. Which approach is best?

- A) Create three separate components: `ActiveBadge`, `InactiveBadge`, `PendingBadge`
- B) Create one `Badge` component with a `status` prop and a color lookup object
- C) Pass the full inline style from the parent each time: `<Badge style={{ background: 'green' }}>Active</Badge>`
- D) Hardcode the color inside each badge using a conditional statement in CSS

Helper Functions

Helper functions are plain JavaScript functions — no React, no hooks, no JSX.

Extract logic to a helper function when it:

- Performs a calculation or data transformation
- Contains no React state or lifecycle
- Could be tested independently
- Might be used in more than one component

```
// utils/cartCalculations.js
export function getCartPrice(cartItems) {
  return cartItems.reduce((total, item) => {
    return total + (item.price * item.itemCount);
  }, 0);
}
```

Place helper functions in `src/utils/` and import them where needed.

Custom Hooks

Custom hooks encapsulate stateful logic that can be shared between components.

Rules:

- Must start with `use` — e.g. `useShoppingCart`, `useLocalStorage`
- Can call other React hooks inside
- Return data and functions — **never JSX**
- Live in `src/hooks/`

When to extract a custom hook:

- Multiple components need the same stateful behavior
- You want to hide complex state management from a component
- Logic involves subscriptions, API calls, timers, or effects

Custom Hooks — Example

```
// hooks/useShoppingCart.js
function useShoppingCart() {
  const [cart, setCart] = useState([]);

  function addItem(product) {
    setCart(prev => [...prev, product]);
  }

  function removeItem(id) {
    setCart(prev => prev.filter(item => item.id !== id));
  }

  return { cart, addItem, removeItem };
}
```

The component calls `useShoppingCart()` and gets back state and updaters — without knowing the implementation details.

Helper Function vs. Custom Hook

	Helper Function	Custom Hook
Name	Any function name	Must start with <code>use</code>
Contains hooks?	No	Yes
Use for	Calculations, filtering, formatting	State management, effects, subscriptions
Where it goes	<code>src/utils/</code>	<code>src/hooks/</code>
Testable without React?	Yes	No (requires a test renderer)

Ask: "Does this logic need `useState` or `useEffect`?" If yes → custom hook. If no → helper function.

Check for Understanding

Which belongs in a custom hook?

- A) A function that formats a price as \$12.99
- B) A function that filters a list of items by category
- C) Logic that fetches user data on mount and stores it in state
- D) A function that converts an array of objects into a CSV string

Project Organization — Five Questions to Ask

Before reorganizing, ask these about each file or section of code:

1. **What feature does it support?** Feature-specific code lives in `features/`.
2. **Is it a layout component?** Headers, footers, sidebars go in `layout/`.
3. **Is it reused across multiple features?** Shared components go in `shared/`.
4. **Is it non-React logic?** Pure utilities go in `utils/`.
5. **Does it use React hooks to share stateful logic?** Custom hooks go in `hooks/`.

Start by answering "what feature?" — most files will fall into place from there.

Project Directory Structure

```
src/
├── features/           ← components grouped by feature
│   ├── ProductList/
│   │   ├── ProductList.jsx
│   │   └── ProductCard.jsx
│   └── Cart/
│       ├── Cart.jsx
│       └── CartItem.jsx
├── shared/             ← cross-feature components
├── layout/             ← Header, Footer
├── hooks/              ← custom hooks
└── utils/              ← helper functions
```

When a feature grows to need sub-components, convert its file to a folder of the same name and move related files inside.

Safe Refactoring Workflow

Move files → update imports → verify → extract components

Step 1: Create the new directory structure.

Step 2: Move files one at a time. For each moved file, update:

- Imports **inside** the moved file (especially non-JS assets like images)
- Imports **in every file that imports it**

Step 3: Keep the dev server running. Let Vite's error overlay catch broken import paths immediately.

Step 4: Extract components — copy JSX into a new file, determine what to pass as props, replace the original with the component instance.

Now: Apply

Let's refactor a working-but-messy component into something scalable.

This Week's Exercise — Refactor a Monolith

Open `src/exercises/lesson-06/StudentWork.jsx`.

The component works — but it does everything in one place:

- Manages multiple pieces of state
- Fetches data
- Filters tasks
- Renders repeated JSX
- Lives in a flat, unorganized project

Your job: **refactor in four steps without changing any behavior**. Run the app after each step to confirm nothing broke.

Exercise — Four Refactoring Steps

Step	Goal
1	Extract reusable components
2	Replace inline JSX with your new components
3	Move filtering logic to a helper function
4	Move data fetching to a custom hook, organize the project

Each step should leave the app in a working state. Commit between steps so you have a recovery point if something goes wrong.

Step 1 — Extract Reusable Components

Create three new files in `src/exercises/lesson-06/components/`. Extract:

- **UserProfile** — the user profile section
- **TaskFilterButtons** — the filter button group
- **TaskItem** — a single task in the list

Each component should:

- Receive everything it needs via props
- Contain no state of its own
- Focus on rendering — no logic

Read `StudentWork.jsx` first and identify which sections repeat or could stand alone before making any changes.

Step 1 (continued) — TaskItem Example

Once you've identified what to extract, create each file and move the JSX in.

```
// components/TaskItem.jsx
function TaskItem({ task }) {
  return (
    <li>
      <strong>{task.title}</strong>
      <span>{task.status}</span>
    </li>
  );
}
export default TaskItem;
```

Follow the same pattern for `UserProfile` and `TaskFilterButtons` — identify what data each needs, pass it as props.

Step 2 — Refactor StudentWork.jsx

Replace the inline JSX with your new components. The app should look and behave exactly the same.

Before

```
{tasks.map(task => (  
  <li key={task.id}>  
    <strong>{task.title}</strong>  
    <span>{task.status}</span>  
  </li>  
))}
```

After

```
import TaskItem from './components/TaskItem';  
  
{tasks.map(task => (  
  <TaskItem key={task.id} task={task} />  
))}
```

Step 3 — Extract a Helper Function

The filtering logic inside `StudentWork.jsx` doesn't need React. Move it out.

Create `src/exercises/lesson-06/utils/filterTasks.js`:

```
export function filterTasks(tasks, filter) {  
  if (filter === 'all') return tasks;  
  return tasks.filter(task => task.status === filter);  
}
```

Use it in `StudentWork.jsx`:

```
import { filterTasks } from './utils/filterTasks';  
  
const visibleTasks = filterTasks(tasks, activeFilter);
```

No hooks, no JSX — just an input and an output. The filtering behavior must stay identical.

Step 4 — Custom Hook + Organize the Project

Move the data-fetching logic to a custom hook and arrange your files.

hooks/useTasks.js

```
function useTasks() {  
  const [tasks, setTasks] = useState([]);  
  const [isLoading, setIsLoading] =  
    useState(true);  
  useEffect(() => {  
    /* fetch logic here */  
  }, []);  
  return { tasks, isLoading };  
}
```

Final structure

```
lesson-06/  
├── components/  
│   ├── TaskItem.jsx  
│   ├── TaskFilterButtons.jsx  
│   └── UserProfile.jsx  
├── hooks/  
│   └── useTasks.js  
├── utils/  
│   └── filterTasks.js  
└── StudentWork.jsx
```

In StudentWork.jsx: `const { tasks, isLoading } = useTasks();`

Wrap-Up

This week you learned:

- A component is reusable when it accepts the same props and has the same structure — variant props and `children` extend that flexibility
- Helper functions hold pure logic (no hooks); custom hooks hold stateful logic (must start with `use`)
- Organize files by asking: feature, layout, shared, utils, or hooks?
- Refactor safely: move one file at a time, update imports, verify with the dev server, commit between steps

Before next session: Complete the weekly assignment — reorganize your todo app into `features/` and `shared/`, extract `TextInputWithLabel` as a reusable component, and add todo editing with local state. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.